Listing 10.8 Promoter without a first and last name field

```
# RiffMates/promoters/models.py
from django.db import models

class Promoter(models.Model):
    common_name = models.CharField(max_length=25)
    full_name = models.CharField(max_length=50)
    famous_for = models.CharField(max_length=50)
```

The migration file for this change has the lengthy name: 0005_remove_promoter_first_name_remove_promoter_last_name.py. You need to run the `migrate` command to have it take effect. Don't forget, the changes you're making need to be reflected in the Django Admin, so you'll need to update your `PromoterAdmin` class if you wrote one.

Database changes in the real world

Depending on the size of your project, how many people are using it, and how many coders are involved, you may need to keep intermediate migration stages alive for long periods of time. Consider the de-Westernization of our `Promoter` model. If our project had an API that accessed `Promoter` or other developers had code that was dependent on `Promoter` containing a first and last name, you may not want to remove those fields immediately. Your production system could stay in stage 0004 for weeks or months, as others catch up to your changes. When you make the intermediary changes, you announce the deprecation of the first and last name fields and a target date for their removal. You only perform the removal after the date has passed and, depending on how much you like your colleagues, after you've verified they're ready for the change. A similar process can be used to rename columns: create a column with the new name, copy the data, keep both columns until the deprecation comes into effect, and then remove the original column.

The `Promoter` case is relatively simple. If the data is only ever accessed through the `Promoter` object, you can write properties that mimic the first and last names without breaking anything. In large, complex systems, there is a chance someone else is integrated directly with your database though, and that requires the columns to continue to exist. This adds the complication of having to maintain consistency across the four name fields. Who said software was easy?

10.3 Squashing a migration

Each change you make to your models means another migration file. If you are frequently standing up new instances of your project from scratch, running all these migrations takes time. You may not need the whole state history of your project, only what things look like now. Django's migration system provides a mechanism for squashing migrations together. It takes two or more migration stages and produces a single file to replace them all.